Introduction à Docker

Sommaire

Introduction à Docker **1**

Installer Docker 2

Créer un conteneur Docker 4

Relier des conteneurs avec Docker Compose 5

Installer Docker

Pour installer Docker, il existe deux possibilités principales : Docker/Rancher Desktop (ou Dockstation sur Linux), et l'installation en CLI (Command Line Interface) via WSL sur Windows. Pour installer WSL, on ouvre PowerShell en mode administrateur et on y saisit les commandes :

- `wsl --set-default-version 2`, pour que votre prochaine installation soit en version 2, plus compatible.
- `wsl --install -d Debian`, pour installer la distribution Debian (ou autre)
- `wsl --set-version Debian 2`, pour s'assurer que la version de votre distribution est bien WSL2.

Une fois WSL installé, on peut désormais installer Docker. On utilise l'utilitaire apt depuis WSL, lancé en mode administrateur. Pour exécuter des commandes avec les droits super utilisateur, on utilise le préfixe « `sudo` » (superuser do). Ensuite, c'est une série de commandes qui installeront Docker, son moteur et ses dépendances :

```
sudo apt-get remove docker docker-engine docker.io containerd runc
sudo apt-get update
sudo apt-get install docker.io
sudo apt-get install \
    ca-certificates \
    curl \
    gnupg \
    lsb-release
sudo mkdir -m 0755 -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/debian/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/debian \
  $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list >
/dev/null
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
sudo service docker start
```

Sur certaines distributions de Linux, on remplace « `service docker start` » par :

```
systemctl start docker
systemctl enable docker
```

Enfin, pour vérifier que docker fonctionne, on exécute l'image de test « hello-world » :

```
sudo docker run hello-world
```

Si tout a bien fonctionné, le terminal devrait afficher ce qui suit :

```
Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

Assurez-vous que le terminal de votre IDE (par exemple, VSCode) soit bien placé dans le dossier « front-app », sinon modifiez le dossier de travail avec la commande « `cd <chemin>` ». Le dossier front-app contient le fichier docker-compose.yaml. Une fois dans celui-ci, c'est la commande « `docker compose up` » qui exécutera son contenu pour lancer les divers conteneurs nécessaires au fonctionnement de l'application. Souvent, ils sont séparés par service (Ex : un conteneur serveur web, un conteneur base de données, etc…). Le stockage persistant des données est garanti par les « volumes » Docker.
Pour lancer le conteneur back-end, répétez les étapes dans le dossier « back-end », cloné précédemment, pour avoir accès aux données via l'interface.

Note : Pour ne pas avoir à utiliser « sudo » systématiquement, vous pouvez également vous rajouter au groupe Docker, en le créant s'il n'existe pas déjà.

```
sudo groupadd docker
sudo usermod -aG docker $USER
```

CREER UN CONTENEUR DOCKER

Pour créer un conteneur Docker, il faut construire (build) une image à partir d'un fichier : Le Dockerfile. C'est ce fichier qui déterminera grâce à des mots clés et des commandes ce que devra contenir et exécuter le conteneur.

Le mot clé « `FROM` » signifie que votre image se construira à partir d'une autre qui existe déjà, et qui sera récupérée depuis le Docker Hub (https://hub.docker.com/). Par exemple, pour créer un projet node, vous pouvez mettre « `FROM node:latest` » où « node » est le nom de l'image et « latest » le tag de la version

Le mot clé « `WORKDIR` » est l'équivalent du « cd » ou « current directory » du terminal, mais à l'intérieur du conteneur. Toutes les autres commandes de Docker seront exécutées depuis celui-ci.

Le mot clé « `COPY` » permet de copier les fichiers d'une source locale à une destination dans le conteneur. On utilise « . » des deux côtés pour copier les fichiers adjacents au Dockerfile (sauf si configuré autrement) dans le WORKDIR du conteneur (Ex : COPY . .).

Le mot clé « `RUN` » permet d'exécuter des commandes pour la construction et le démarrage du conteneur, et celles-ci seront exécutées depuis le WORKDIR. Si l'image est en cache, ces commandes pourraient ne pas s'exécuter.

Le mot clé « `CMD` » permet également d'exécuter une commande, mais ne fait pas partie des étapes de construction du conteneur. S'il y a plusieurs CMD dans un Dockerfile, seule la dernière sera exécutée. Il est recommandé d'utiliser RUN pour la construction & l'installation et CMD pour le démarrage.

Un exemple de Dockerfile (généré à l'aide de GitHub Copilot et de commentaires):

Dockerfile U

Dockerfile > ...

```
#Create a node Dockerfile starting from the latest node image
FROM node:latest
#Create a directory for the app
RUN mkdir -p /usr/src/app
#Set the working directory to the app directory
WORKDIR /usr/src/app
#Copy the package.json file to the app directory
COPY package.json /usr/src/app/
#Install the dependencies
RUN npm install
#Copy the rest of the app files to the app directory
COPY . /usr/src/app
#Expose the port 3000
EXPOSE 3000
#Start the app
CMD ["npm", "start"]
```

Relier des conteneurs avec Docker Compose

En règle générale, on cherche à séparer chaque service (Serveur web, BDD, …) dans un conteneur distinct pour faciliter leur gestion. Pour articuler ces conteneurs et les relier les uns aux autres pour faire fonctionner une application, on utilise Docker Compose. C'est le fichier docker-compose.yml qui permet cette configuration. Chaque service doit être nommé et configuré dans ce fichier.

```yaml
version: '3.9'
services:
    # Liste des services utilisés pour cette composition Docker
    #Service Web
    web:
        build:
            context: ./
            dockerfile: ./web/nginx/web.dockerfile
        image: mydocker-web
        container_name: mydocker-web
        restart: always
        volumes:
            # Config nginx
            - type: bind
              source: "./web/nginx/conf/custom/"
              target: "/etc/nginx/custom/"
              consistency: consistent
            # Fichiers web statiques
            - type: bind
              source: "./www/"
              target: "/var/www/html/"
              consistency: consistent
            # Logs nginx
            - type: bind
              source: "./web/nginx/logs/"
              target: "/var/log/nginx/"
              consistency: consistent
        ports:
            - "80:80"
        links:
            - php
        depends_on:
            - php
        networks:
            - mydocker-network
        command: [ "nginx", "-g", "daemon off;" ]
```

Dans cet exemple, après avoir donné la version de Docker Compose, on définit un premier service serveur web avec nginx. On peut choisir de partir directement d'une image du Docker Hub (comme avec FROM) ou d'utiliser un Dockerfile. Ici, pour la construction de l'image, on utilisera le Dockerfile dans le sous-dossier web/nginx/ :

```dockerfile
nginx > web.dockerfile > ...
# Image de base
# Alpine est une version allégée de Linux
# qui permet de réduire la taille de l'image
FROM nginx:stable-alpine
# Copier les fichiers de configuration de Nginx
COPY ./web/nginx/conf/ /etc/nginx/
# Copier les fichiers du site
COPY ./www/ /var/www/html
# Exposer l'application sur le port 80
EXPOSE 80
```

On précise ensuite que ce Dockerfile doit agir comme s'il se trouvait dans le même dossier que docker-compose.yaml, grâce à « `context: ./` ».

Ce Dockerfile copiera donc les fichiers statiques et de configuration et créera une image et un conteneur au nom de « `mydocker-web` ». La directive « restart » permet de redémarrer le conteneur automatiquement en cas de plantage. Lorsque le conteneur n'est plus actif, les données modifiées ne sont pas conservées. C'est pourquoi il faut utiliser des volumes.

Pour monter un volume dans un conteneur, on assigne (`bind`) un dossier source sur l'hôte qui pointera sur un dossier cible (`target`) à l'intérieur du conteneur. Pour que la synchronisation soit instantanée, on paramètre `consistency` à « `consistent` ». Pour des raisons de performances, on peut également utiliser « `cached` » pour que le dossier hôte soit en mode lecture seule pour le conteneur et applique les changements avec un délai (recommandé pour le code), ou « `delegated` » pour que ce soit le conteneur qui soit en lecture seule et qui écrive dans l'hôte avec un délai (recommandé pour les logs). À noter qu'il peut parfois être nécessaire de relier un même volume à plusieurs conteneurs, notamment avec nginx et php qui ont tous les deux besoin de voir les fichiers statiques pour les lire. Si le dossier/fichier hôte d'un volume n'existe pas, des erreurs peuvent survenir.

Ensuite, pour rendre le conteneur visible depuis l'extérieur, il faut mapper un port d'accès par rapport à celui dans le conteneur. Par exemple, si on héberge un serveur web avec le port 80 dans le conteneur et qu'on souhaite y accéder depuis notre machine hôte avec le port 81 (si l'hôte a déjà un autre serveur web sur 80 par exemple), on paramètrera « `port : 81:80` ».

Pour exprimer une dépendance entre conteneurs, on utilise « `depends_on` » pour s'assurer qu'un conteneur n'est pas exécuté sans un autre. Dans d'anciennes versions, on utilisait « `links` » pour lier plusieurs conteneurs et s'assurer qu'ils communiquent sur le même réseau.

Ici, « `networks` » précise que le conteneur communique sur « `mydocker-network` ». Ce réseau doit être configuré au bas du fichier docker-compose.yaml pour pouvoir être utilisé communément par d'autres conteneurs, identifiés par un alias DNS qui porte le nom de leur service respectif:

```
networks:
    mydocker-network:
        driver: bridge
```

Ainsi, web peut voir le conteneur php avec l'alias « php ».

Enfin, c'est le paramètre « `command` » détermine la commande à exécuter lorsque le conteneur est prêt à démarrer.

Pour construire et démarrer l'ensemble des conteneurs du fichier docker-compose.yaml, on saisira la commande « `docker compose up` ». Par défaut, lorsqu'une image est construite, son contenu est stocké sur la machine pour ne pas avoir à la reconstruire intégralement à chaque exécution. C'est pourquoi la commande « `docker compose up` » peut être suivie d'arguments qui permettent de supprimer les conteneurs, les images et de recommencer leur construction depuis le début, notamment pour s'assurer que tous les changements en développement soient bien appliqués dans le conteneur.

Cet extrait de script permet à un utilisateur de choisir s'il souhaite tout reconstruire avant l'exécution ou s'il souhaite exécuter sa composition de façon standard :

```
# Si l'utilisateur a saisi Y ou y
if [[ "$choix" == [Yy] ]]; then
    # Script pour démarrer le conteneur compose sans laisser de traces et supprimer les précédents
    sudo docker compose rm -f
    sudo docker compose down
    # Recréer les conteneurs de chaque service sans cache
    sudo docker compose build --no-cache
    # Démarrer le docker-compose en forçant la reconstruction/la mise à jour des images et la recréation des conteneurs
    sudo docker compose up --pull --build --always-recreate-deps --force-recreate --no-deps --remove-orphans
# Si l'utilisateur a saisi N ou n
elif [[ "$choix" == [Nn] ]]  || [ -z $choix]; then
    # Script pour démarrer le conteneur compose normalement
    sudo docker compose up
fi
```

On peut également supprimer des conteneurs avec docker image prune et docker image rm.